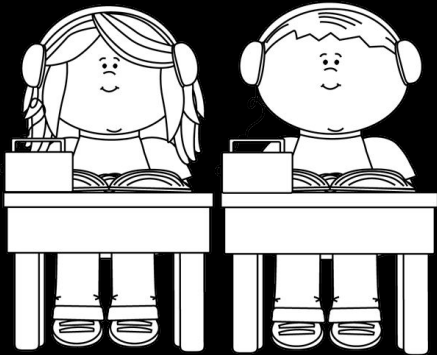


Jesús Jara
Phd student UAM



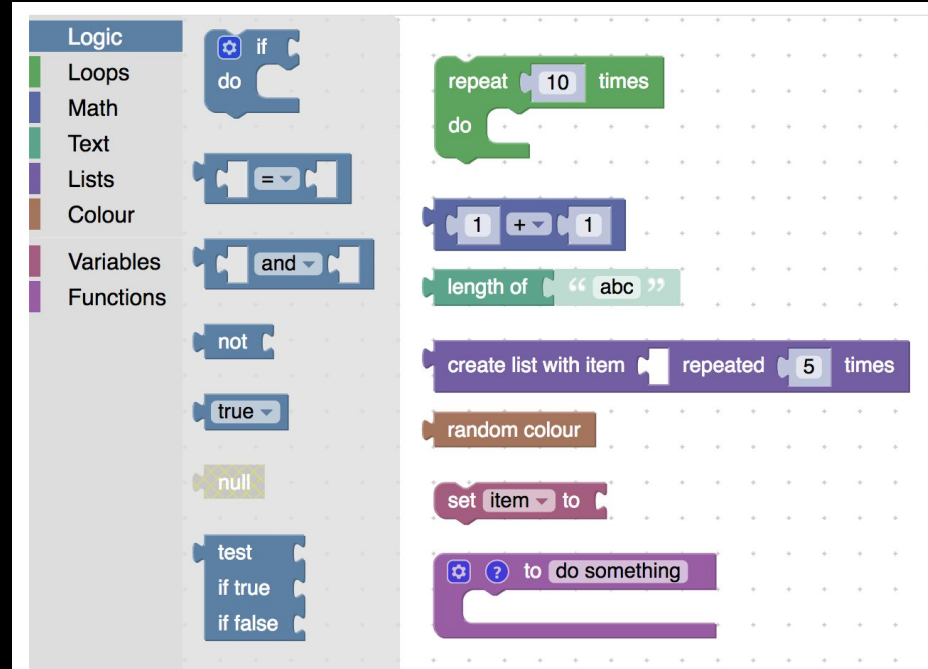
*Blockly, Web Audio API and
ready-made Blocks for music
teaching with code*

Workshop materials:

<https://github.com/LaVerbenaElectronica/blockly-music>

What is Blockly?:

Blockly is a *visual*, drag-and-drop programming tool developed by Google. It allows users to create programs by snapping together visual blocks, which represent different coding commands and concepts.



Code generation:

Can generate code in various programming languages, including JavaScript, Python, and PHP.

Educational tool:

Widely used in education to introduce programming concepts to children and adults.

Open-Source:

Free and open-source software, allowing developers to modify and extend its functionality.

Behind platforms like:

Scratch, APP Inventor, Microsoft CODE, Micro:bit, or Ozo Blockly, among other projects are built with Blockly.

What is Tone.js?:

Tone.js is a Web Audio framework for creating interactive music in the browser. The architecture of Tone.js aims to be familiar to both musicians and audio programmers creating web-based audio applications.

```
1 // create two monophonic synths
2 const synthA = new Tone.FMSynth().toDestination();
3 const synthB = new Tone.AMSynth().toDestination();
4 //play a note every quarter-note
5 const loopA = new Tone.Loop((time) => {
6   |   synthA.triggerAttackRelease("C2", "8n", time);
7   | }, "4n").start(0);
8 //play another note every off quarter-note, by starting it "8n"
9 const loopB = new Tone.Loop((time) => {
10  |   synthB.triggerAttackRelease("C4", "8n", time);
11  | }, "4n").start("8n");
12 // all loops start when the Transport is started
13 Tone.getTransport().start();
14 // ramp up to 800 bpm over 10 seconds
15 Tone.getTransport().bpm.rampTo(800, 10);
```

DAW features:

It allows synchronising and scheduling sound events.

Modular design:

It promises to be flexible enough to design sounds in a modular-way, by connecting objects.

Prebuilt synths:

It comes with a set of predefined synths that can be used out of the box.

Various Effects:

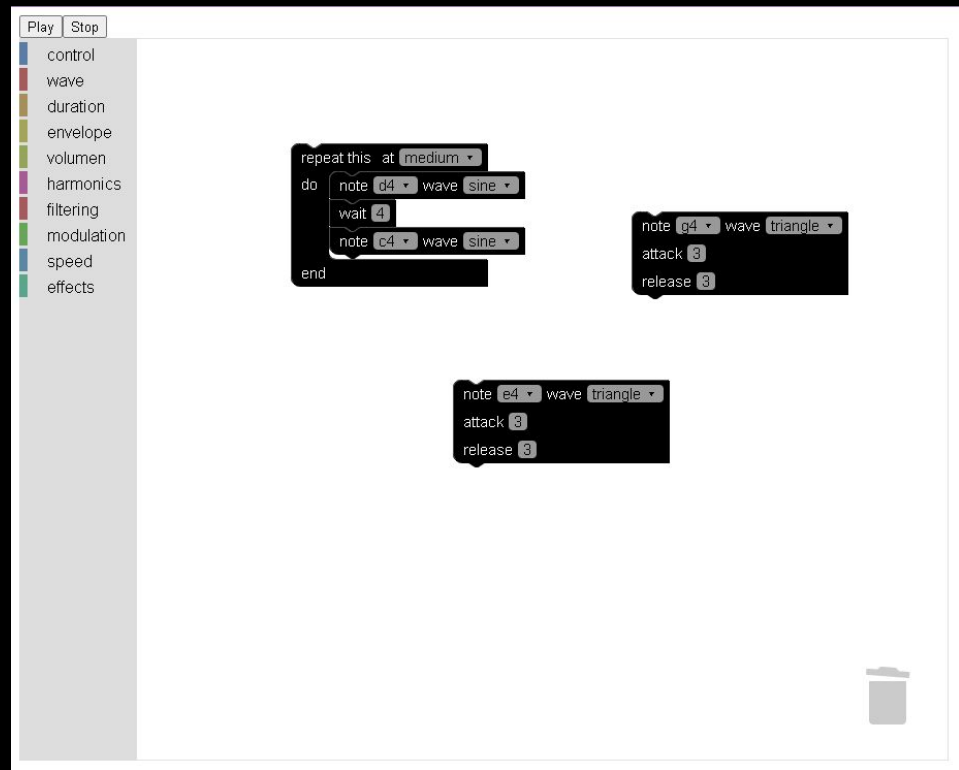
It allows effects like filters, envelopes, reverb, delay or distortion among others.

What is in the [github](#) repo?:

Inspired by [Toneblocks](#) (Quigley & Payne, 2021). This repo is a [standalone](#) work-in-progress app that uses Blockly to generate [Tone.js](#) code.

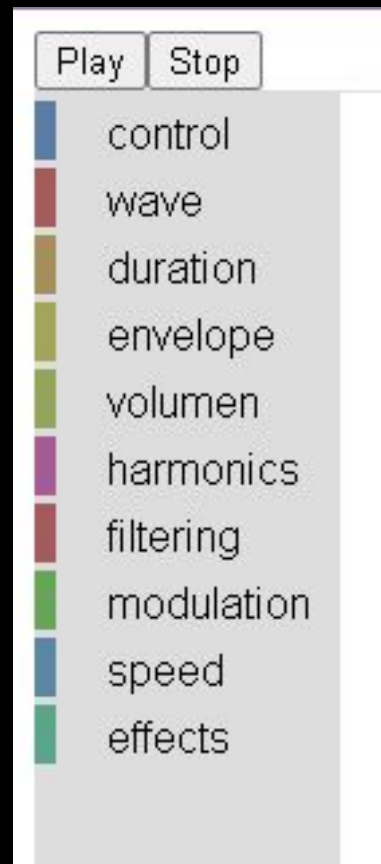
My goal is to show how to [design](#) blocks so that [everyone](#) can create their own blocks specially teachers.

And yes, it is still very [buggy](#).



Design decisions:

Blocks are organised in categories that make sense for musical curriculum instead of computer science curriculum.



Design decisions:

Blocks designs are repetitive.

There are different ways to express the same computer instruction, allowing students to review the same concepts again and again and to focus on one thing at a time.



note c4 wave sine



note c4 wave sine dur 1



note c4 wave sine dur 1 vol 1

Design decisions:

Blocks designs can be more creative.

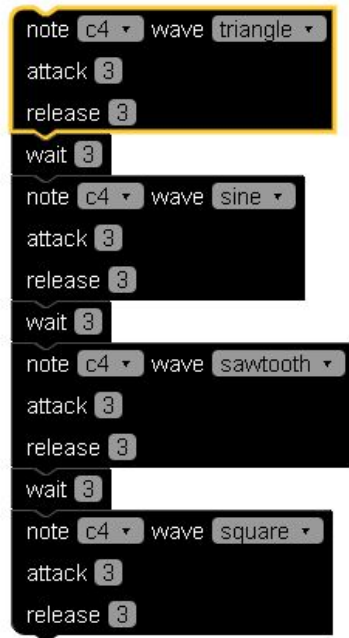
They allow the designer to explore a vocabulary closer to the child imagination, making the learning process more meaningful.



Design decisions:

The combination of both categories and blocks allow to imagine small études of music snippets for the students to practice.

Helping teachers to explore both a new music curriculum and the computer science curriculum.



Ready to begin?

Workshop materials:

`app.js` *//This file for the implementation*

`index.html` *//This file for the interface*

`Tonejs-tests.html` *//This file to test javascript code*

Interface: index.html

```
<!DOCTYPE html>
<html>

<head>
  <!-- libraries -->
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/blockly/3.20200924.4/blockly_compressed.min.js"
integrity="sha512-QXLWks3csNuoy9T6AoZm8BewWPHVGUsz0HcWpTF6xH8Vv55GxI4h4h0+ghQHfdmv7nnm9GV4I5Y1kK
OWWStf6g=="crossorigin="anonymous"></script>

    <!-- Load the Tone library -->
    <script src="https://unpkg.com/tone"></script>
    <!-- Load the proper app -->
    <script defer src="app.js"></script>
</head>

<body>
  <button id="play-btn">Play</button><button id="stop-btn">Stop</button>
  <div id="blocklyDiv" style="height: 700px; width: 900px;"></div>
</body>
</html>
```

Application: app.js

Play button and runCode function: *These functions are at the beginning*

Toolbox: *The toolbox categories are specified from line 38*

Block definitions: *The definitions appear from line 160*

Block implementation: *The implementations are from 271*

Toolbox: `app.js` → *they are defined as json structures, more on that [here](#).*

```
const toolbox = {
  kind: 'categoryToolbox',
  contents: [
    {
      kind: "category", name: "control", colour: "212",
      contents: [
        {
          "kind": 'block',
          "type": 'loop',
        },
        {
          "kind": "block",
          "type": "math_number",
          "fields": {
            "NUM": 1
          }
        },
        {
          "kind": "block",
          "type": "wait",
          "fields": {
            "NUM": 1
          }
        }
      ]
    },
  ],
},
```


Block definition: `app.js` → *Let's create a new block using this tool [here](#).*

 **Blockly**

Create new blockLoad blockDelete

Input

Fields

Connection Checks

Colour

name my_block

inputs

value input NAME

fields leftcheckbox ✓, NAMElabel create your own custom blocks

connection check any

automatic inputs

no connections

tooltip""

help url""

colourhue: 225°



Preview

✓ create your own custom blocks

Output Configuration ⓘ

Block definition format: ☒ JSON ☐ JavaScript

Blockly import format: ☒ Import (esm/cjs) ☐ Script tags

Code generator language:

Code Headers ⓘ

import * as Blockly from 'blockly/core';
import {javascriptGenerator, Order} from 'blockly/javascript';

Block Definition ⓘ

```
{  
  "type": "my_block",  
  "tooltip": "",  
  "helpUrl": "",  
  "message0": "%1 create your own custom blocks %2",  
  "args0": [  
    {  
      "type": "field_checkbox"
```

Generator Stub ⓘ

```
javascriptGenerator.forBlock['my_block'] = function() {  
  const checkbox_name = block.getFieldValue('NAME');  
  // TODO: change Order.ATOMIC to the correct operator pre
```

Block implementation: app.js → More on this [here](#).

Basic synth in Tone.js:

```
//create a synth and connect it to the main output (your speakers)
const synth = new Tone.Synth().toDestination();
//play a middle 'C' for the duration of an 8th note
synth.triggerAttackRelease("C4", "8n");
```

Setters in Tone.js:

```
const synth = new Tone.Synth().toDestination();
const now = Tone.now();
synth.set({oscillator: {type: 'sine'}});
synth.set({envelope: {attack: 0.1, decay: 0.15, sustain: 1, release: 0.1}});
synth.triggerAttackRelease("C4", "8n", now + 1);
```

Block implementation: `app.js` → More on this [here](#).

Basic effects in Tone.js:

```
const filter = new Tone.Filter(400, "lowpass").toDestination();
const feedbackDelay = new Tone.FeedbackDelay(0.125, 0.5).toDestination();
player.connect(filter);
player.connect(feedbackDelay);
```

Samplers in Tone.js:

```
const player = new
Tone.Player("https://tonejs.github.io/audio/berklee/gong_1.mp3").toDestination();
Tone.loaded().then(() => {player.start();});
```

Block implementation: app.js → More on this [here](#).

How to evaluate Tone.js code in Blockly:

```
Blockly.JavaScript['sound_wave_envelope'] = function (block) {  
  const note = block.getFieldValue('note');  
  const waveType = block.getFieldValue('wavetype');  
  const attack = block.getFieldValue('attack');  
  const release = block.getFieldValue('release');  
  const dur = 1;  
  const code = `const synth` + num + ` = new Tone.Synth().toDestination();  
  synth` + num + `.set({oscillator: {type: '${waveType}'}});  
  synth` + num + `.set({envelope: {attack: '${attack}', decay: 0.15, sustain: '${dur}', release:  
  '${release}'}});  
  synth` + num + `.triggerAttackRelease('${note}', `+ (attack + dur + release) + `, now + ` +  
timeDur + `);`;  
  num++;  
  return code;  
};
```

Block implementation: app.js → More on this [here](#).

Example of a new Block definition:

```
Blockly.Blocks['play_note'] = {  
  init: function() {  
    this.appendValueInput('Freq')  
      .setCheck('Number')  
      .appendField('play note of frequency: ');  
    this.setInputsInline(true);  
    this.setPreviousStatement(true, null);  
    this.setNextStatement(true, null);  
    this.setTooltip('use a math field to connect on  
it');  
    this.setHelpUrl('');  
    this.setColour(330);  
  }  
};
```

Example of a new Block implementation:

```
Blockly.JavaScript['play_note'] = function (block) {  
  var value =  
    Blockly.JavaScript.valueToCode(block, 'Freq',  
    Blockly.JavaScript.ORDER_ATOMIC)  
  
  const dur = 1;  
  const code = `const synth` + num + ` = new  
Tone.Synth().toDestination();  
  synth` + num + `.set({oscillator: {type: 'sine'}});  
  synth` + num + `.triggerAttackRelease(${value}, 1,  
0);`;   
  num++;  
  return code;  
};
```

Thanks